

SWE-Explore: Benchmarking How Coding Agents Explore Repositories

Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, Xiaodong Gu,
llmXive-implementer-v1.0

ABSTRACT

Repository-level coding benchmarks such as SWE-bench have driven a rapid surge in the capabilities of coding agents. Yet they usually treat coding tasks as a holistic, binary prediction problem (e.g., resolved or unresolved), neglecting fine-grained agent capabilities such as repository understanding, context retrieval, code localization, and bug diagnosis. In this paper, we introduce SWE-Explore, a benchmark that isolates the evaluation of repository exploration, a critical capability of coding agents. Given a repository and an issue, SWE-Explore asks an explorer to return a ranked list of relevant code regions under a fixed line budget. SWE-Explore covers 848 issues across 10 programming languages and 203 open-source repositories. For each instance, we derive line-level ground truth from independent agent trajectories that successfully solved the same issue, distilling the specific code regions their solution paths actually consulted. We evaluate exploration along coverage, ranking, and context-efficiency dimensions, showing that these metrics strongly track downstream repair behavior. Across a broad set of retrieval methods, general coding agents, and specialized localizers, we find that agentic explorers form a clear tier above classical retrieval. While file-level localization is already strong for modern methods, line-level coverage and efficient ranking remain the key axes differentiating state-of-the-art explorers.

About this document

This is a **Reviewed Preprint**: a third-party paper that llmXive has *auto-reviewed* (with its LLM review panel) but *never modified*. The original manuscript follows this cover page **exactly as published** by its authors — llmXive re-typesets nothing and claims no authorship.

Provenance. Ingested by llmXive on 2026-07-02 — scraped from arXiv; via llmXive dashboard, GitHub issue #297; submitted by github-actions[bot].

Source. <https://arxiv.org/abs/2606.07297>

About llmXive. llmXive is an automated platform for open scientific discovery. Its specialist LLM agents advance their own ideas from brainstorm to peer-reviewed paper, and they also review notable third-party papers — drawn from the most-downloaded papers on Hugging Face and

from submissions by human contributors. Every submitted paper is reviewed by the LLM panel *and* used to seed a new llmXive follow-up study. This paper is one such third-party work: llmXive reviewed it but did not write it. Learn more at <https://github.com/ContextLab/llmXive>.

Automated review. See the accompanying [peer-review-llmxive.pdf](#) for the full reviewer report.

llmXive follow-up. A separate llmXive study building on this work: PROJ-897-llmxive-follow-up-extending-swe-explore.

SWE-Explore: Benchmarking How Coding Agents Explore Repositories

Shaoqiu Zhang^{1*}, Yuhang Wang^{1*}, Jialiang Liang^{2*}, Yuling Shi¹, Wenhao Zeng¹,
Maoquan Wang⁴, Shilin He⁵, Ningyuan Xu⁴, Siyu Ye³, Kai Cai⁴, Xiaodong Gu^{1†}

¹Shanghai Jiao Tong University ²Xinjiang University

³University of Illinois at Urbana-Champaign ⁴Independent Researcher

⁵The Chinese University of Hong Kong

 <https://github.com/Qiushao-E/SWE-Explore-Bench>
 <https://huggingface.co/datasets/SWE-Explore-Bench/SWE-Explore-Bench>

Abstract

Repository-level coding benchmarks such as SWE-bench have driven a rapid surge in the capabilities of coding agents. Yet they usually treat coding tasks as a holistic, binary prediction problem (e.g., resolved or unresolved), neglecting fine-grained agent capabilities such as repository understanding, context retrieval, code localization, and bug diagnosis. In this paper, we introduce **SWE-Explore**, a benchmark that isolates the evaluation of repository exploration, a critical capability of coding agents. Given a repository and an issue, SWE-Explore asks an explorer to return a ranked list of relevant code regions under a fixed line budget. SWE-Explore covers 848 issues across 10 programming languages and 203 open-source repositories. For each instance, we derive line-level ground truth from independent agent trajectories that successfully solved the same issue, distilling the specific code regions their solution paths actually consulted. We evaluate exploration along coverage, ranking, and context-efficiency dimensions, showing that these metrics strongly track downstream repair behavior. Across a broad set of retrieval methods, general coding agents, and specialized localizers, we find that agentic explorers form a clear tier above classical retrieval. While file-level localization is already strong for modern methods, line-level coverage and efficient ranking remain the key axes differentiating state-of-the-art explorers.

1 Introduction

Repository-level coding benchmarks, such as SWE-bench [11], have driven a rapid surge in the capabilities of automated coding agents [6, 7, 33]. The ecosystem around these benchmarks has expanded quickly: new evaluation distributions now cover multilingual repositories, multimodal software issues, and harder, long-horizon professional tasks [2, 29, 30, 32]. In parallel, scalable training-oriented resources like SWE-smith [30], SWE-Gym [15], and SWE-Dev [23] are actively fueling agent development. Supported by these robust resources, frameworks such as SWE-agent [28], AutoCodeRover [34], Agentless [27], OpenHands [24], Claude Code [1], Mini-SWE-Agent [28], and AweAgent [3] have successfully turned repository-scale issue resolution into a practical, everyday testbed for software engineering agents.

However, the widespread adoption of these benchmarks stems from a protocol that is both their strength and primary limitation: each repair attempt is reduced to a single pass/fail prediction, as

*Equal contribution

†Corresponding author: Xiaodong Gu, xiaodong.gu@sjtu.edu.cn

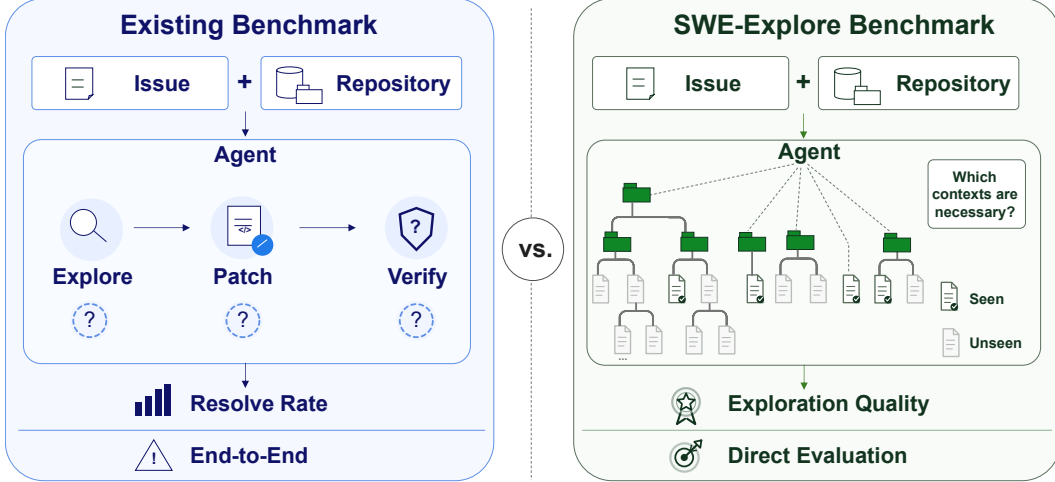


Figure 1: Motivation of SWE-Explore. A holistic metric of resolution rate conflates exploration, localization, and patch synthesis. SWE-Explore isolates repository exploration as a line-level evaluation target.

shown in Figure 1. While this binary metric makes models directly comparable, it obscures the underlying mechanics of success. A holistic pass/fail score cannot reveal which specific step—reading the relevant code, localizing the bug, generating the patch, or validating the fix—actually succeeded or failed. Once we step back from this single prediction, two distinct failure modes emerge. An agent either fails to explore the relevant code for the fix, or it retrieves sufficient evidence but fails to synthesize a correct patch. While the latter is readily captured by existing executable benchmarks, the former is largely hidden. A real-world repository contains thousands of files. Determining which specific lines carries the evidence for a given issue is a daunting challenge, even for the agents that ultimately solve it. This decomposition is increasingly recognized by recent work on context management and bug localization [5, 13, 22, 26, 31, 37].

Consequently, the capability of coding agents in repository exploration remains under-measured. Despite recent efforts to study agentic localization and retrieval [5, 13, 22, 26, 31, 35, 37], existing evaluations still lack a common, precise target for comparing classical retrievers, search agents, and long-context selectors. Measuring file or function level localization only indicates whether an agent reached the right general neighborhood, no metric reveals exactly *which* lines of code were explored. Without visibility into line-level coverage, we cannot rigorously evaluate how well an agent explores a repository before it begins to write code.

In this paper, we introduce **SWE-Explore**, a benchmark that turns repository exploration into a comparable evaluation target. Given an issue and a repository, an explorer is asked to return a ranked list of code regions; we then score that list against ground truth derived from independent agent trajectories that successfully solved the same issue, asking how early it surfaces the evidence those trajectories actually relied on. The output format is deliberately simple: sparse retrievers, interactive agents, and long-context selectors are all compared as producers of the same ranked region list under a fixed line budget. This lets SWE-Explore evaluate exploration behavior without requiring the explorer to write or validate a patch.

To check that a higher exploration score actually leads to better repair, SWE-Explore is paired with a controlled downstream protocol: we feed each explorer’s output—and only that output—as the available repository context to a fixed coding agent, and measure whether the resulting patch passes the original test suite. This protocol is not a replacement for the primary benchmark but a way to verify that what our exploration metrics measure is the same thing that drives downstream success. In this sense, the downstream protocol serves as an external validity check, while the benchmark itself remains a lightweight context-selection task.

In summary, our contributions are as follows:

- **A new evaluation target.** We isolate repository exploration from end-to-end repair and formalize it as a ranked, line-level context selection task, so that retrievers, search agents, and long-context selectors can be compared on the axis they are designed to improve.

Table 1: Comparison of SWE-Explore with existing repository-level coding and exploration benchmarks across six design dimensions covering ground-truth granularity, evaluation protocol, and ranked-region assessment.

Benchmark	Exec. Based	Multi- Lingual	Line-Level GT	Trajectory- Grounded GT	Joint Expl. + Repair Eval	Ranked Region Eval
Loc-Bench [5]	✗	✗	✗	✗	✗	✗
SWE-bench Verified [6, 11]	✓	✗	✗	✗	✗	✗
SWE-bench Multilingual [30]	✓	✓	✗	✗	✗	✗
SWE-bench-Pro [7]	✓	✓	✗	✗	✗	✗
ContextBench [13]	✓	✓	✗	✗	✓	✗
SWE-ContextBench [37]	✓	✗	✗	✗	✗	✗
SWE-Explore (Ours)	✓	✓	✓	✓	✓	✓

- **Trajectory-grounded supervision.** We propose a novel method to annotate ground truth lines from successful agent trajectories, with least requiring manual annotation.
- **A studied metric set, validated against repair.** We systematically compare coverage, ranking, and budget-efficiency metrics, and use a controlled downstream protocol—where each explorer’s output is the only context visible to a fixed coding agent—to show that the metrics we keep are predictive of repair success across a broad set of explorers.

2 Related Work

2.1 Coding Benchmarks

Repository-level benchmarks have established executable issue resolution as the central evaluation target for coding agents. SWE-bench couples issue descriptions, repository snapshots, and harness-based verification [11], with Verified [6] and Live [33] variants tightening quality and contamination control. Subsequent work broadens the scope along several axes: multilingual coverage (SWE-bench Multilingual[30], Multi-SWE-bench[32]), multi-turn and rebased settings (SWE-bench Multimodal[29], SWE-rebench[2]). A second line of work pushes evaluation toward intermediate behavior: ContextBench [13] introduces human-annotated gold contexts and scores retrieval over agent trajectories [13], while SWE-Pruner [26] and SWE-ContextBench [37] respectively benchmark context compression and experience reuse. Related efforts also examine adjacent repository-level abilities, including hierarchical debugging and correctness checking, programmer behavior patterns, multi-agent debate, experience-driven repair, and repository-level question answering [4, 12, 16, 19, 21].

These benchmarks either target the full issue-to-patch pipeline or evaluate isolated facets of intermediate behavior; what is missing is a single benchmark in which trajectory-grounded, line-level exploration quality and its downstream effect on issue resolution can be measured jointly, as shown in Table 1. This matters because exploration quality is not fully captured by either coarse context labels or final resolve rate: an explorer may reach the right file but miss the decisive span, or surface the right evidence too late in a ranked output. SWE-Explore is complementary rather than competing: it derives line-level supervision directly from successful agent trajectories, evaluates exploration as a ranked region-list task, and pairs the exploration score with a restricted-context executable validation on the same instances.

2.2 Explorer Methods

Localization and explorer methods study how relevant code is found inside a repository. Classical retrieval and bug-localization work ranks code artifacts from natural-language reports, with TF-IDF [18] and BM25 [17] as lightweight baselines and IR-based bug localization targeting likely faulty files [36]. Semantic code search and repository retrieval broaden this setting to natural-language-to-function search and cross-file code completion [8, 14], while nDCG [9] provides a standard way to reward useful evidence appearing early in a ranked list. Long-context compression work further highlights that selecting, compressing, and preserving code context are themselves central design

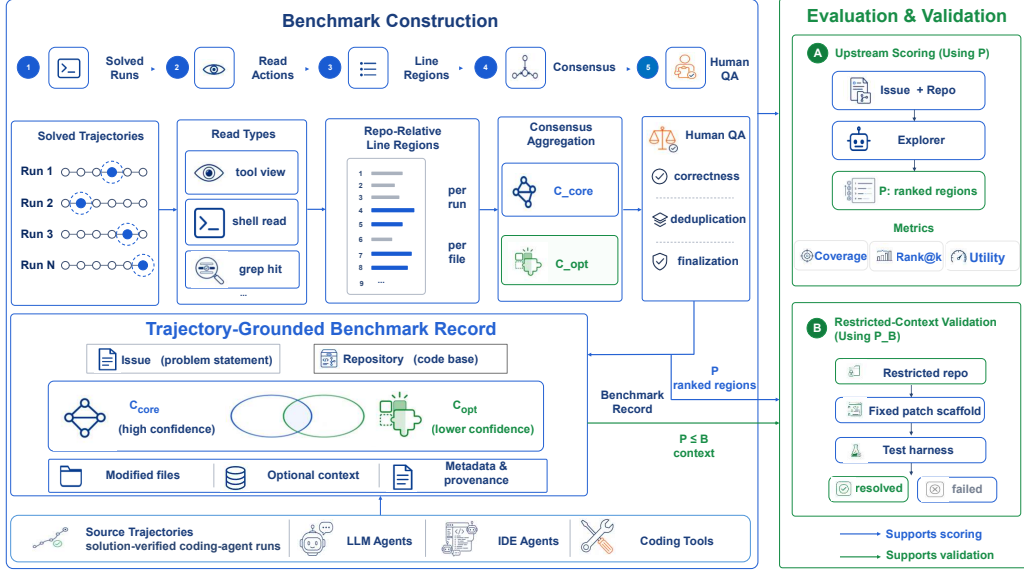


Figure 2: Overview of SWE-Explore. From solution-verified trajectories, SWE-Explore extracts read actions, aggregates them into core and optional context, and evaluates explorers using both upstream exploration metrics and downstream restricted-context validation.

choices for code models and agents [20, 25]. These evaluations supply useful methodology, but their targets are usually query–snippet relevance, next-line completion relevance, or bug-file relevance rather than the line regions consulted during successful issue resolution.

Recent LLM-based methods move from static retrieval toward interactive exploration. AutoCodeRover combines LLM reasoning, code search, and program analysis [34]; LocAgent [5], OrcaLoca [31], and CoSIL [10] evaluate localization over files, functions, ranked entities, or iterative code-graph search; and CodeScout studies pre-exploration for problem-statement improvement [22]. General-purpose coding agents further show that navigation, context management, tool use, and patch generation are tightly coupled in practical repair [1, 3, 24, 27, 28]. What remains less systematic is a common target for comparing lexical retrievers, dense retrievers, rerankers, and agentic explorers as ranked, line-level region producers. SWE-Explore fills this gap with trajectory-grounded line-level targets and a fixed-scaffold downstream protocol in which only the selected context varies.

3 SWE-Explore Benchmark

3.1 Task Formulation

SWE-Explore formulates *repository exploration* as a standalone functionality. Given an issue q and a repository snapshot $\mathcal{R}$, SWE-Explore returns a ranked list of relevant code regions:

$$f : (q, \mathcal{R}) \mapsto P = (r_1, r_2, \dots, r_K),$$

where $P = (r_1, r_2, \dots, r_K)$ is the *ranked region list*. Each region $r_i = (p_i, s_i, e_i)$ consists of a file path p_i and a line range $[s_i, e_i]$. SWE-Explore does not require a final patch, does not access the ground truth, and is not required to interact with the repository. As shown in Figure 2, the benchmark scores P against trajectory-grounded supervision derived per instance (§3.3) using the metric family in §3.4. Independently, we use a restricted-context repair bridge (§3.4) as a one-time methodological validation that these metrics track repair behavior; the bridge is not part of the standard evaluation loop, and a new explorer can be benchmarked using only the metrics above.

3.2 Data Sources

SWE-Explore is built on three public repository-level data sources: SWE-bench Verified [6, 11], SWE-bench-Pro [7], and SWE-bench Multilingual [30]. After the solution-verification filter described in §3.3, we retain 848 instances spanning 10 programming languages and 203 open-source repositories.

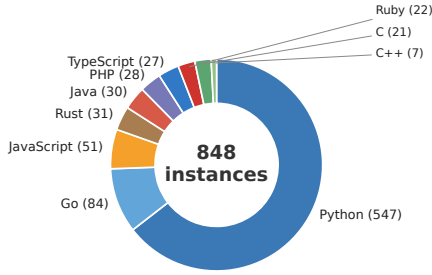


Figure 3: Language distribution of the 848 retained SWE-Explore instances across 10 different coding languages.

Table 2: Per-instance averages of the ground-truth core context $|R_{\text{core}}|$ at the file, region, and line granularity.

		Mean	Max
Issue Text	Length (Words)	191.2	1,892
Ground-Truth Context	# Files	4.3	15
	# Regions	4.7	15
	# Lines	1,578	16,136
Provenance	# Source Trajectories	2.9	4
	# Modified-by-Patch Files	1.4	66
Codebase	# Files (non-test)	759	7,649
	# Lines (non-test)	179.6K	1.4M

As Table 2 and Figure 3 summarizes, each instance carries on average 4.3 ground-truth files, 4.7 regions, and 1,578 visible lines, embedded in repositories that average 759 files and 180K lines of non-test source code. Per-source breakdowns and the full benchmark distribution are deferred to Appendix A.

3.3 Ground-Truth Annotation

SWE-Explore keeps only instances for which we observe *at least two successful issue-resolution trajectories* from strong LLMs such as GPT-5.4, Gemini-3-Pro, Sonnet-4.6, GLM-5.1, and Kimi-K2.6. Instances without two successful trajectories are excluded, because their trajectory-derived context would not support the cross-run agreement signal used below. After this filter, 848 instances are retained across the three source datasets.

Trajectory Source. Directly annotating necessary context by hand is costly at our scale and difficult to make consistent: hundreds of repository-level issues span ten languages, and annotators may draw different boundaries around helpers, configuration, and tests. SWE-Explore instead derives supervision from *solution-verified agent trajectories*: successful runs by strong coding agents such as GPT-5.4, Gemini-3-Pro, Sonnet-4.6, GLM-5.1, and Kimi-2.6 under the original harness. For each retained instance, we collect its successful trajectory set T with $|T| \geq 2$.

We treat regions repeatedly surfaced across independent successful trajectories as a behavioral signal for core context: they are the code spans that different solution paths naturally explored while resolving the same issue. Given T , we first intersect read regions across trajectories to obtain conservative core candidates, then use an LLM-based refinement step to promote a small subset of model-specific optional reads when they are load-bearing for the issue. Finally, the authors manually audit every refined ground truth against the issue and trajectories, removing unsupported regions.

Extracting reads. From each trajectory we collect all read actions that resolve to an explicit file–interval pair—editor-style view tool calls, command-line reads (`cat/head/tail/sed -n`), and `grep -n` line hits within $\mathcal{R}$ —and normalize them into regions (p, s, e) . Actions we cannot unambiguously map to such a pair (e.g., free-form terminal interaction) are discarded rather than heuristically expanded, keeping the supervision record strictly grounded in observable reads.

Generating regions. Let $R(\tau)$ be the set of (p, s, e) regions extracted from trajectory τ as described above. We first compute a conservative intersection candidate $R_{\text{int}} = \bigcap_{\tau \in T} R(\tau)$ and collect model-specific optional reads outside this intersection as $R_{\text{opt}}^{(m)} = (\bigcup_{\tau \in T_m} R(\tau)) \setminus R_{\text{int}}$, where $T_m \subseteq T$ is the successful trajectories from model family m . Intersection and union are taken file-wise at the line level, so two overlapping reads of `parser.py:40-80` and `parser.py:60-100` contribute `parser.py:60-80` to R_{int} .

The final ground-truth core R_{core} used in this paper is the *refined* version of R_{int} : an LLM-based refinement step promotes a small subset of optional reads when they are load-bearing for the issue, and the authors then manually audit the resulting regions. Unless otherwise stated, all later uses of

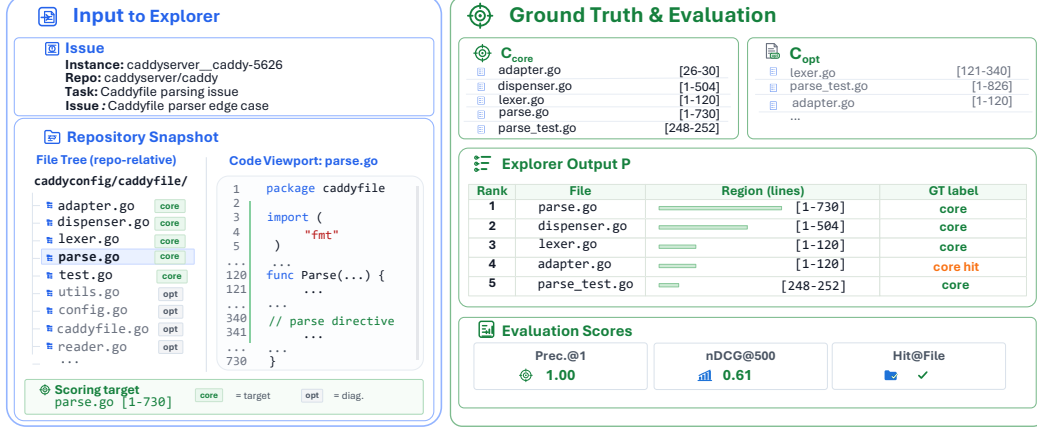


Figure 4: Example of a SWE-Explore instance. **Left:** an issue plus a repo snapshot with the highlighted core span. **Right:** trajectory-derived core regions C_{core} (scoring target) and optional regions C_{opt} , an explorer’s ranked prediction scored against C_{core} .

R_{core} refer to this refined and audited ground truth, and R_{core} is the only scoring target in the main experiments. Figure 4 shows an example instance of SWE-Explore Bench. Details and ablations comparing pure intersection, refined core, and union variants are deferred to Appendix B.

3.4 Metrics

With ground truth in hand, the next question is how to score an explorer’s ranked region list against it. Let $L(r) \subseteq \{(p, \ell)\}$ be the set of (p, ℓ) pairs covered by region r , and for budget B let $P_{\leq B}$ be the longest prefix of P whose cumulative $|L(\cdot)|$ does not exceed B . Write $L(P) = \bigcup_i L(r_i)$ and $Y = L(R_{\text{core}})$.

Coverage and accuracy. *Precision* and *recall* are defined at the line level, $\text{PREC} = |L(P) \cap Y|/|L(P)|$ and $\text{REC} = |L(P) \cap Y|/|Y|$, with F1 their harmonic mean. We also report two coarser hit rates that capture the practically important event of overlapping the right code even when line spans are imprecise: a file-level $\text{HITFILE} = |\{p : \exists i, p_i = p\} \cap \text{FILES}(Y)|/|\text{FILES}(Y)|$, and a region-level analogue $\text{HITREGION} = |\{r \in R_{\text{core}} : \exists i, L(r_i) \cap L(r) \neq \emptyset\}|/|R_{\text{core}}|$, which counts the fraction of core regions for which the explorer surfaced at least one overlapping prediction.

Ranking under budget. We adapt nDCG to a line-budget setting. Each predicted region r_i is assigned a gain g_i equal to the number of core lines it covers; regions are processed in their predicted order, and $\text{DCG}@B$ accumulates the discounted gain over the longest prefix whose cumulative line count stays within the budget B :

$$\text{DCG}@B = \sum_{i \in P_{\leq B}} \frac{g_i}{\log_2(i+2)}.$$

$\text{nDCG}@B$ normalizes this against the best DCG attainable on the same instance under the same line budget; the ideal-ordering procedure is described in Appendix C. Using a line budget instead of a rank cutoff means a single verbose region that exhausts the budget without proportional gain is penalized just as heavily as omitting useful content. We additionally report *first useful hit* (FUH), defined as $1 - i^*/|P|$ where i^* is the smallest rank whose visible lines intersect the core target Y (and 0 if no rank in P does); higher FUH means the explorer surfaced useful evidence earlier.

Efficiency and noise. *Context efficiency* is the fraction of predicted visible lines that fall inside $L(R_{\text{core}}) \cup L(R_{\text{opt}}^{(m)})$, quantifying how much of the selected context is grounded evidence versus off-target. A complementary *noise rate*, defined as the fraction of predicted regions overlapping neither R_{core} nor $R_{\text{opt}}^{(m)}$, serves as a region-level diagnostic.

Table 3: Downstream resolve rate under the restricted-context validation environment (GPT-5.4 with Mini-SWE-Agent, $K=5$).

Explorer	Resolve Rate (%)
Oracle	59.7
Random	4.7
TF-IDF	26.0
RAG	23.3
BM25	12.7
CoSIL	59.3
Mini-SWE-Agent	50.0
Openhands	47.7
OrcaLoca	45.3
AutoCodeRover	44.7
LocAgent	44.7
AweAgent	41.3
Codex	50.3
Claude Code	48.0

Table 4: Explorer-level correlation between each upstream exploration metric and downstream resolve rate, computed across all explorers in our pool. $\downarrow$ marks lower-is-better.

Metric	Pearson r	Spearman ρ
CtxEff	+0.950	+0.739
FUH	+0.928	+0.675
Rec@100	+0.926	+0.845
HitFile	+0.925	+0.695
nDCG@500	+0.921	+0.460
nDCG@300	+0.920	+0.458
nDCG@100	+0.917	+0.480
HitReg	+0.901	+0.695
Prec	+0.890	+0.671
NoiseReg $\downarrow$	-0.812	-0.562
NoiseFile $\downarrow$	-0.808	-0.590
Rec@300	+0.769	+0.796
Rec@500	+0.710	+0.796
F1	+0.673	+0.810
Rec $_{\ell}$	+0.617	+0.796

Validation by downstream repair. To check that the metrics above track downstream repair behavior, we construct a one-time *restricted-context environment*: for a given explorer output P , we hide everything in the repository outside $\bigcup_i (p_i, [s_i, e_i])$, and ask a fixed coding agent to produce a patch that is then judged by the original SWE-bench harness. This is a one-time sanity check on the metrics and is not part of the standard evaluation procedure, and §4.2 uses it to quantify how well each metric predicts resolve rate; full implementation details—the sanitized container, the line-budget value B , the test-callback interface, and the patch scaffold—are given in Appendix D.

4 Experiments

4.1 Setup

Explorers. We evaluate explorers from four families. Two *baselines* bound the dynamic range: Oracle returns R_{core} directly, and Random returns uniformly sampled regions. *Sparse retrievers* are represented by BM25 [17] and TF-IDF [18]. As a *lightweight dense retriever* we use a RAG pipeline instantiated with Potion, a static word-embedding retriever distilled from a sentence transformer. Finally, the *agentic explorers* cover five general-purpose coding agents (Claude Code [1], Codex, OpenHands [24], Mini-SWE-Agent [28], AweAgent [3]) and four published academic localization agents (AutoCodeRover [34], LocAgent [5], OrcaLoca [31], CoSIL [10]); Oracle in particular plays a central role in validating our ground-truth construction (§4.2).

Metrics. Following the analysis in §3.4, we report a combination of strongly predictive metrics and standard baselines. The primary metrics are Precision, nDCG@500, HitFile, and Context Efficiency, selected for their high correlation with downstream behavior and low mutual redundancy. We additionally report Recall, F1, and hit/noise region rates as conventional references, even though several of these exhibit weaker predictive power individually. Full metric definitions are in §3.4.

Choice of K . On our refined ground truth, the per-instance number of core regions averages roughly 4.7 after the LLM-refinement step (§3.3). We therefore fix $K=5$ for every explorer in this paper: each explorer is asked to return its five most relevant regions, which keeps the comparison both fair across systems and aligned with the size of the supervision target.

4.2 Downstream validation.

Before using upstream exploration metrics as the main evaluation target, we ask whether they track downstream repair behavior. On a shared $n=150$ subset of SWE-Explore Bench, each explorer

Table 5: Exploration quality at $K=5$ across different LLMs powering the same **Mini-SWE-Agent** scaffold. **Bold** marks the best result per column; underline marks the second best.

Model	Coverage & Accuracy					Ranking			Efficiency & Noise	
	HitReg	Prec	Rec _ℓ	F1	HitFile	nDCG@500	Rec@500	FUH	CtxEff	NoiseReg↓
GPT-5.4	<u>0.516</u>	0.542	<u>0.154</u>	<u>0.194</u>	0.655	<u>0.905</u>	<u>0.154</u>	<u>0.927</u>	0.771	0.258
GPT-5.4-mini	0.531	0.509	0.185	0.215	<u>0.649</u>	0.924	0.183	0.956	<u>0.754</u>	<u>0.265</u>
Kimi-K2.6	0.413	0.475	0.117	0.149	0.509	0.739	0.115	0.759	0.676	0.316
Sonnet-4.5	0.428	<u>0.519</u>	0.118	0.154	0.535	0.779	0.116	0.802	0.715	0.279
GLM-4.7	0.289	0.414	0.122	0.148	0.343	0.557	0.105	0.572	0.536	0.465
Gemini-3-Pro	0.268	0.420	0.052	0.079	0.369	0.605	0.052	0.620	0.540	0.467

Table 6: Exploration quality at $K=5$. **Bold** marks the best non-oracle result per column; underline marks the second best. HitReg / HitFile are region-/file-level hit rates; Rec_ℓ is line-level recall. ↓ indicates lower is better; all others are higher-is-better. All agentic explorers are driven by GPT-5.4 as the underlying model.

Explorer	Coverage & Accuracy					Ranking			Efficiency & Noise	
	HitReg	Prec	Rec _ℓ	F1	HitFile	nDCG@500	Rec@500	FUH	CtxEff	NoiseReg↓
Oracle	0.915	1.000	0.953	0.964	0.923	0.858	0.576	1.000	1.000	0.000
Random	0.003	0.002	0.004	0.002	0.004	0.004	0.001	0.006	0.002	0.997
BM25	0.065	0.055	0.021	0.024	0.079	0.132	0.021	0.141	0.087	0.910
TF-IDF	0.121	0.117	0.049	0.054	0.140	0.223	0.049	0.240	0.190	0.821
Potion	0.069	0.055	0.025	0.026	0.088	0.136	0.025	0.146	0.100	0.897
OpenHands	0.514	0.489	0.179	0.209	0.645	0.867	0.177	0.895	0.737	0.245
Mini-SWE-Agent	0.505	0.530	0.151	0.190	0.640	0.885	0.151	0.907	0.754	0.253
AweAgent	<u>0.534</u>	0.577	0.140	0.182	0.682	0.954	0.140	<u>0.975</u>	<u>0.829</u>	0.191
AutoCodeRover	0.272	0.680	<u>0.233</u>	<u>0.291</u>	0.280	0.720	0.165	0.730	0.738	<u>0.034</u>
LocAgent	0.472	<u>0.642</u>	0.191	0.241	0.540	<u>0.950</u>	0.173	0.977	0.799	0.195
OrcaLoca	0.126	0.295	0.033	0.049	0.129	0.311	0.030	0.313	0.317	0.003
CoSIL	0.544	0.581	0.788	0.602	0.544	0.824	0.412	0.920	0.898	0.471
Claude Code	0.531	0.598	0.154	0.202	<u>0.667</u>	0.938	0.154	0.963	<u>0.829</u>	0.186
Codex	0.516	0.523	0.194	0.223	0.649	0.901	<u>0.190</u>	0.936	0.762	0.249

provides its $K=5$ ranked regions; the resulting context is then given to a fixed Mini-SWE-Agent patcher backed by GPT-5.4 and Gemini-3-Pro, and we average the SWE-bench harness resolve rate over the two patchers. Table 4 reports the explorer-level Pearson and Spearman correlations between each upstream metric and this downstream resolve rate.

Stable signals. The strongest metrics are not purely file-level or purely recall-based. Context Efficiency has the highest Pearson correlation ($r=0.950$), suggesting that useful context must be both relevant and compact. Rec@100 is the strongest rank-correlated signal ($\rho=0.845$), indicating that early coverage under a tight line budget is especially predictive of repair. HitFile, HitRegion, and FUH remain strong Pearson signals: they capture whether the explorer reaches the right file, overlaps the right evidence, and surfaces useful evidence early. Precision is also predictive, but it is most informative when interpreted together with coverage-oriented metrics.

Useful but incomplete signals. Rank-aware metrics such as nDCG@500 have very high Pearson correlation but weaker Spearman correlation, indicating that they separate broad quality tiers well but are less stable for ordering nearby explorers. Recall-style metrics show a budget effect: Rec@100 is strong in both views, while broader recall metrics and F1 are more rank-sensitive than scale-sensitive, reflecting their tendency to reward broad reading even when the selected context is not compact. Noise rates have the expected negative correlation, but serve better as diagnostics than as standalone success measures. Together, these results justify reporting a mixed metric set rather than a single score. In Table 6 and Table 5, we therefore emphasize HitRegion, HitFile, Precision, nDCG@500, FUH, and Context Efficiency, while retaining Recall, F1, Recall@500, and noise-region rate as complementary diagnostics.

4.3 Exploration Quality

Table 5 and Table 6 report upstream exploration quality for all explorers and models at $K=5$. We highlight the following observations.

Agentic exploration is a clear step above non-agentic retrieval. Sparse retrievers (BM25, TF-IDF) and the lightweight dense retriever remain close to Random on most metrics, while every agentic explorer is substantially higher than them. This confirms that repository exploration is not well captured by one-shot lexical or embedding retrieval alone: multi-step interaction with the repository is already necessary to reach the metric range occupied by modern coding agents.

Low F1 is mostly a recall problem. Despite strong file-level hit rates and ranking scores, most non-oracle explorers still have low F1, and the limiting term is usually line-level recall rather than precision. The general-purpose coding agents all reach high HitFile and nDCG@500, but their Rec_ℓ remains only around 0.14–0.19; AutoCodeRover is highly precise, yet also recall-limited. This suggests that broad enough repository exploration remains a central bottleneck for current code-agent-style explorers: they often find plausible files early, but miss many of the specific spans needed to cover the full ground-truth context.

LLM choice shifts the operating point, but not the bottleneck. Table 5 controls the scaffold by running the same Mini-SWE-Agent explorer with different LLMs. The GPT-family models form the strongest tier, but with slightly different profiles: GPT-5.4 is cleaner and more compact, while GPT-5.4-mini surfaces more core regions and ranks useful evidence earlier. Kimi-K2.6 and Sonnet-4.5 form a middle tier, and GLM-4.7 and Gemini-3-Pro lag mainly in coverage and ranking. The larger pattern is more important than the exact ordering: across all LLMs, file-level hits remain much stronger than line-level recall, so replacing the base model alone does not remove the exploration bottleneck. High-recall region discovery still appears to require better exploration mechanisms, not just a stronger patching model.

General coding agents behave surprisingly similarly. Claude Code, Codex, OpenHands, Mini-SWE-Agent, and AweAgent have closely matched profiles across coverage, ranking, and efficiency metrics. This is notable because they differ in implementation and harness complexity, yet their exploration outputs occupy nearly the same operating point: high file hit, high early ranking, compact context, and low line recall. The similarity suggests that a complex repair harness is not necessarily required to study the exploration subproblem; a simpler explorer interface can expose much of the same behavior.

Specialized localizers only help when they broaden search. The academic agents do not uniformly dominate general coding agents. AutoCodeRover is precise but conservative, OrcaLoca has very low noise but misses many relevant regions, and LocAgent resembles the general-agent profile rather than changing the recall frontier. CoSIL is the main exception: it achieves by far the highest non-oracle Rec_ℓ and F1, suggesting that its iterative code-graph search is an important component for high-recall exploration. In contrast, explorers that rely more heavily on shell-style navigation or narrow search actions may reach the right files while still under-covering line-level evidence.

Line-level evaluation adds information beyond file hits. HitFile remains a strong and useful signal, as shown by the downstream correlations in §4.2; however, it does not distinguish whether an explorer actually surfaces the relevant spans inside those files. The contrast between high HitFile and much lower Rec_ℓ across most agentic explorers supports SWE-Explore’s line-level design: file-level localization captures reaching the right neighborhood, while line-level metrics measure whether the evidence needed by successful trajectories is actually exposed.

4.4 Controlled Context Degradation

The previous sections show that explorer behavior differs along coverage, precision, ranking, and efficiency, and that these differences affect downstream repair. Here we ask a more controlled robustness question: is a patcher more sensitive to *missing* relevant context or to *redundant* irrelevant context? In the restricted-context validation environment (§3.4), we synthetically perturb Oracle context by exposing only an $\alpha \in \{0, 25, 50, 75, 100\}$ fraction of core regions (missing-context

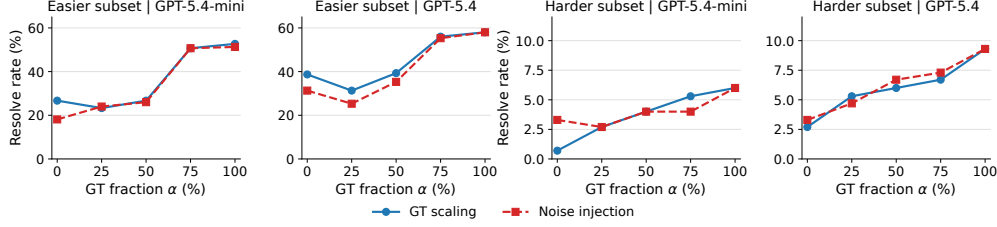


Figure 5: Resolve rate as the visible context degrades from the Oracle’s full core set R_{core} to either $\alpha\%$ of R_{core} alone (*GT scaling*, solid) or $\alpha\%$ of R_{core} padded back to full size with random non-core regions (*noise injection*, dashed).

condition), or by filling the removed budget with randomly sampled non-core regions (redundant-context condition). We sweep α on two stratified $n=150$ subsets of SWE-Explore under both a weak patcher GPT-5.4-mini and a strong patcher GPT-5.4; Figure 5 reports all four panels.

Missing context is the dominant failure mode. On the easier subset, downstream resolve rate changes sharply only after enough core evidence is present: performance stays low through partial context, then jumps between $\alpha=50$ and $\alpha=75$. This threshold-like pattern suggests that patchers are not simply accumulating value smoothly from every additional region; instead, several pieces of core evidence must be present together before a correct fix becomes likely. Redundant context is less damaging once this threshold is crossed: the redundant-context curve closely tracks the missing-context curve for $\alpha \geq 75$, indicating that modern patchers can tolerate extra irrelevant code when the essential evidence is already visible. This agrees with the metric analysis above: in the high-coverage regime, missing core evidence matters more than moderate precision loss, so recall-oriented improvements are more valuable than small filtering gains. Redundancy hurts most when core evidence is scarce, especially at $\alpha=0$, where random non-core code lowers resolve rate by 7–9 pp. The harder subset shows a much narrower range, suggesting that when the issue itself exceeds the patcher’s capability, improving context alone has limited effect.

The easier-subset dip suggests caution with empty-context baselines. Both easier-subset curves dip from $\alpha=0$ to $\alpha=25$, especially under the stronger patcher. A plausible explanation is memorization: with no repository context, the model may rely on an issue-only prior, while a small incomplete slice of R_{core} pushes it to reconcile partial evidence. Because the dip disappears on the harder subset, we treat it as a caveat rather than the main effect: empty-context baselines on canonical repositories may be inflated and should be interpreted carefully.

5 Conclusion

We presented **SWE-Explore**, a benchmark for evaluating repository exploration independently from patch generation through ranked, line-level context selection. Using trajectory-derived supervision, SWE-Explore compares retrievers, search agents, and long-context selectors by the evidence they surface rather than only by final repair outcomes. Our experiments show that exploration metrics track downstream repair, that current agents are strong at finding relevant files but remain recall-limited at the line level, and that missing core evidence hurts more than moderate redundant context. We hope SWE-Explore provides a focused target for building explorers that read repositories more broadly and expose the spans repair agents actually need.

References

- [1] Anthropic. Claude code: Ai-assisted coding in real-world codebases, 2025. URL <https://claude.ai/code>. Accessed: 2026-05.
- [2] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents, 2025. URL <https://arxiv.org/abs/2505.20411>.
- [3] Guoxin Chen, Fanzhe Meng, Jiale Zhao, Minghao Li, Daixuan Cheng, Huatong Song, Jie Chen, Yuzhi Lin, Hui Chen, Xin Zhao, Ruihua Song, Chang Liu, Cheng Chen, Kai Jia, and Ji-Rong

- Wen. Beyondswe: Can current code agent survive beyond single-repo bug fixing?, 2026. URL <https://arxiv.org/abs/2603.03194>.
- [4] Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, et al. Swe-exp: Experience-driven software issue resolution. *arXiv preprint arXiv:2507.23361*, 2025.
 - [5] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor K. Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. *CoRR*, abs/2503.09089, 2025. doi: 10.48550/arXiv.2503.09089.
 - [6] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench verified, August 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>. OpenAI milestone, updated February 24, 2025.
 - [7] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?, 2025. URL <https://arxiv.org/abs/2509.16941>.
 - [8] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. CodeSearchNet challenge: Evaluating the state of semantic code search, 2019. URL <https://arxiv.org/abs/1909.09436>.
 - [9] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418.
 - [10] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. Issue localization via llm-driven iterative code graph searching, 2025. URL <https://arxiv.org/abs/2503.22424>.
 - [11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
 - [12] Han Li, Yuling Shi, Shaoxin Lin, Xiaodong Gu, Heng Lian, Xin Wang, Yantao Jia, Tao Huang, and Qianxiang Wang. Swe-debate: Competitive multi-agent debate for software issue resolution. *arXiv preprint arXiv:2507.23348*, 2025.
 - [13] Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. ContextBench: A benchmark for context retrieval in coding agents. *CoRR*, abs/2602.05892, 2026. doi: 10.48550/arXiv.2602.05892.
 - [14] Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=pPjZIOuQuF>.
 - [15] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym, 2025. URL <https://arxiv.org/abs/2412.21139>.
 - [16] Weihan Peng, Yuling Shi, Yuhang Wang, Xinyun Zhang, Beijun Shen, and Xiaodong Gu. Swe-qa: Can language models answer repository-level code questions? *arXiv preprint arXiv:2509.14635*, 2025.
 - [17] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019.

- [18] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988. doi: 10.1016/0306-4573(88)90021-0.
- [19] Yuling Shi, Songsong Wang, Chengcheng Wan, Min Wang, and Xiaodong Gu. From code to correctness: Closing the last mile of code generation with hierarchical debugging. *arXiv preprint arXiv:2410.01215*, 2024.
- [20] Yuling Shi, Yichun Qian, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. Longcodezip: Compress long context for code language models. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 141–153. IEEE, 2025.
- [21] Yuling Shi, Hongyu Zhang, Chengcheng Wan, and Xiaodong Gu. Between lines of code: Unraveling the distinct patterns of machine and human programmers. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 1628–1639. IEEE, 2025.
- [22] Manan Suri, Xiangci Li, Mehdi Shojai, Songyang Han, Chao-Chun Hsu, Shweta Garg, Aniket Anand Deshmukh, and Varun Kumar. CodeScout: Contextual problem statement enhancement for software agents. *CoRR*, abs/2603.05744, 2026. doi: 10.48550/arXiv.2603.05744.
- [23] Haoran Wang, Zhenyu Hou, Yao Wei, Jie Tang, and Yuxiao Dong. SWE-dev: Building software engineering agents with training and inference scaling. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 3742–3761. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-acl.193. URL <https://aclanthology.org/2025.findings-acl.193/>.
- [24] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents, 2024. URL <https://arxiv.org/abs/2407.16741>.
- [25] Yifei Wang, Ziteng Wang, Yuling Shi, Silin Chen, Xinrui Wang, Yueqi Wang, Beijun Shen, Linjing Li, Xiaodong Gu, Julian McAuley, et al. Context compression for llm agents: A survey of methods, failure modes, and evaluation. 2026.
- [26] Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. SWE-pruner: Self-adaptive context pruning for coding agents. *CoRR*, abs/2601.16746, 2026. doi: 10.48550/arXiv.2601.16746.
- [27] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *CoRR*, abs/2407.01489, 2024. doi: 10.48550/arXiv.2407.01489.
- [28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=mXpq6ut8J3>.
- [29] John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, Diyi Yang, Sida Wang, and Ofir Press. SWE-bench multimodal: Do AI systems generalize to visual software domains? In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=riTiq3i21b>.
- [30] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Swe-smith: Scaling data for software engineering agents, 2025. URL <https://arxiv.org/abs/2504.21798>.
- [31] Zhongming Yu, Hejia Zhang, Yujie Zhao, Hanxian Huang, Matrix Yao, Ke Ding, and Jishen Zhao. OrcaLoca: An LLM agent framework for software issue localization. In *International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=LyUfPOvM6I>.

- [32] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and Liang Xiang. Multi-swe-bench: A multilingual benchmark for issue resolving, 2025. URL <https://arxiv.org/abs/2504.02605>.
- [33] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. Swe-bench goes live!, 2025. URL <https://arxiv.org/abs/2505.23419>.
- [34] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024. doi: 10.1145/3650212.3680384.
- [35] Zejun Zhang, Jian Wang, Qingyun Yang, Yifan Pan, Yi Tang, Yi Li, Zhenchang Xing, Tian Zhang, Xuandong Li, and Guoan Zhang. MULocBench: A benchmark for localizing code and non-code issues in software projects, 2025. URL <https://arxiv.org/abs/2509.25242>.
- [36] Jian Zhou, Hongyu Zhang, and David Lo. Where should the bugs be fixed? more accurate information retrieval-based bug localization based on bug reports. In *Proceedings of the 34th International Conference on Software Engineering*, pages 14–24. IEEE, 2012. doi: 10.1109/ICSE.2012.6227210.
- [37] Jared Zhu, Minhao Hu, and Junde Wu. SWE context bench: A benchmark for context learning in coding. *CoRR*, abs/2602.08316, 2026. doi: 10.48550/arXiv.2602.08316.

A Dataset Details

This appendix supplements §3.2. The main paper reports the aggregate benchmark statistics; here we specify the retained instance set, record schema, and repository-snapshot assumptions used by SWE-Explore.

Source benchmarks and filtering. SWE-Explore is constructed from three public repository-level sources: SWE-bench Verified, SWE-bench-Pro, and SWE-bench Multilingual. We keep an instance only when at least two trajectory resolves the original task under the source benchmark’s executable harness. This filtering step ensures that the supervision target is extracted from successful repair behavior rather than from failed exploration attempts. After filtering, SWE-Explore contains 848 instances across 10 programming languages and 203 open-source repositories.

Benchmark composition. The retained set combines Python-centered verified issues, harder professional software-engineering tasks, and multilingual issue-resolution tasks. This design keeps the benchmark tied to executable repair while reducing dependence on a single language ecosystem or repository family.

Language distribution. The language distribution is shown in Table 7. Python remains the largest subset because SWE-bench Verified is Python-centric; SWE-bench-Pro and SWE-bench Multilingual add most of the non-Python coverage.

Benchmark record schema. Each instance is stored as a structured record containing the issue, repository metadata, trajectory provenance, and line-level supervision. The core fields are:

- `instance_id`: unique instance identifier;
- `repo`: source repository name;
- `source`: source benchmark;
- `problem_statement`: natural-language issue description;
- `ground_truth.read_core_regions`: line-level core regions used for scoring;
- `ground_truth.read_optional_regions`: optional regions used for diagnostics and context-efficiency computation;

Table 7: Language distribution of SWE-Explore.

Language	Instances	Percentage (%)
Python	547	64.5
Go	84	9.9
JavaScript	51	6.0
Rust	31	3.7
Java	30	3.5
PHP	28	3.3
TypeScript	27	3.2
Ruby	22	2.6
C	21	2.5
C++	7	0.8
Total	848	100.0

- **provenance:** successful source trajectories and extraction metadata.

All file paths are stored as repository-relative paths. Line intervals are 1-indexed and closed. Before scoring, paths are canonicalized so that equivalent spellings such as `./src/foo.py` and `src/foo.py` map to the same repository-relative file.

Repository snapshots. Each instance is evaluated against a fixed repository snapshot inherited from its source benchmark. The same snapshot is used to resolve line intervals, score explorer predictions, and run restricted-context downstream validation. Generated files, temporary files, external dependency paths, and files outside the repository checkout are not treated as valid repository files.

B Ground-Truth Construction and Refinement Details

This appendix supplements §3.3. The main paper describes the trajectory-grounded construction at a high level; here we record the extraction, normalization, refinement, and audit rules used to produce the final line-level targets.

Read extraction. We extract observable file-reading behavior and convert it into repository-relative line regions. We parse three types of read signals:

- **Editor views:** tool calls with an explicit file path and visible line range;
- **Command-line reads:** commands such as `cat`, `head`, `tail`, and `sed -n` when the target file can be resolved;
- **Search hits:** `grep -n` outputs that can be mapped to repository files and line numbers.

Signals that cannot be mapped to a unique file–interval pair are discarded. This rule keeps the supervision tied to observable repository reads and avoids expanding ambiguous terminal output into unsupported line regions.

Path normalization. Absolute paths are accepted only when they point inside the repository checkout. Relative paths are normalized by removing redundant `./` segments, resolving `..` segments whenever possible, and matching the result against the repository file index. Reads mapped to multiple candidate files are discarded. Reads mapped outside the repository are also discarded.

Line-interval normalization. Each read is converted into a tuple (p, s, e) , where p is the repository-relative path and $[s, e]$ is a 1-indexed closed interval. Whole-file reads are expanded using the file’s line count at the evaluated checkout. Out-of-range intervals are clipped to valid file boundaries. Empty intervals are removed. Adjacent or overlapping intervals from the same trajectory and file are merged before cross-trajectory aggregation.

Core and optional context. Let $\mathcal{T}$ be the successful trajectory set for an instance and let $R(\tau)$ denote the merged line regions read by trajectory τ . The raw core context is the file-wise, line-level intersection across successful trajectories:

$$R_{\text{core}}^{\text{raw}} = \bigcap_{\tau \in \mathcal{T}} R(\tau).$$

The raw optional context is the portion of successful reads outside this intersection:

$$R_{\text{opt}}^{\text{raw}} = \left(\bigcup_{\tau \in \mathcal{T}} R(\tau) \right) \setminus R_{\text{core}}^{\text{raw}}.$$

The union has high recall but includes exploratory detours, redundant file openings, and model-specific context that may not be necessary for repair. The intersection is more conservative: it keeps only evidence consulted across successful solution paths. SWE-Explore therefore uses the refined core as the main scoring target and keeps optional context for diagnostics and context-efficiency computation.

LLM-assisted refinement. Pure intersection can under-cover cases where different successful agents use different but equivalent evidence. We therefore consider optional regions that are repeatedly visited, directly adjacent to core evidence, or close to modified regions. For each candidate, the refinement model receives the issue statement, the candidate region, nearby code context, and a compact summary of successful trajectories. The output schema contains:

- a binary decision on whether the candidate is load-bearing;
- a short rationale grounded in the issue and trajectory evidence;
- the precise line interval to promote into the refined core.

Candidates without a precise promoted interval are rejected.

Human audit. Every promoted region is manually audited against the issue, source trajectories, and final patch. The audit checks whether:

- the region exists in the repository at the evaluated checkout;
- the region is relevant to the issue rather than merely adjacent or frequently opened;
- including it rewards evidence that a successful solution plausibly relied on.

Regions that fail any check are removed. The audit keeps the target conservative while recovering load-bearing context that pure intersection may miss.

Context variants. For analysis, we maintain three target variants: pure intersection, refined core, and full union. The pure-intersection target is maximally conservative. The full-union target is high-recall but noisy. The refined-core target balances these two extremes and is the default target used in the main experiments.

C Metric Definitions and Ideal-Order Implementation

This appendix supplements §3.4. The main paper defines the metric family used in the experiments; here we give the evaluator-level definitions, including duplicate handling, budgeted prefixes, and the ideal-order computation used by nDCG.

Line universe. All metrics are computed over repository-relative line identifiers (p, ℓ) , where p is a normalized path and ℓ is a 1-indexed line number. A predicted region contributes all visible lines in its clipped interval. Duplicate predicted lines are counted once for set-based precision and recall, while the original region order is preserved for rank-aware metrics.

Let $L(r)$ denote the set of line identifiers covered by region r , let $L(P) = \bigcup_i L(r_i)$ denote the union of predicted lines, and let $Y = L(R_{\text{core}})$ denote the line-level core target.

Coverage metrics. Line-level precision and recall are defined as:

$$\text{Prec} = \frac{|L(P) \cap Y|}{|L(P)|}, \quad \text{Rec}_\ell = \frac{|L(P) \cap Y|}{|Y|}.$$

F1 is the harmonic mean of precision and recall:

$$\text{F1} = \frac{2 \cdot \text{Prec} \cdot \text{Rec}_\ell}{\text{Prec} + \text{Rec}_\ell}.$$

Hit rates. We also report two coarser hit rates. HITFILE measures whether the prediction reaches ground-truth files:

$$\text{HitFile} = \frac{|\{p : \exists i, p_i = p\} \cap \text{Files}(Y)|}{|\text{Files}(Y)|}.$$

HITREGION measures whether each ground-truth region is overlapped by at least one predicted region:

$$\text{HitRegion} = \frac{|\{r \in R_{\text{core}} : \exists i, L(r_i) \cap L(r) \neq \emptyset\}|}{|R_{\text{core}}|}.$$

Budgeted prefixes. For a line budget B , $P_{\leq B}$ is the longest prediction prefix whose cumulative visible lines do not exceed B . This prefix definition penalizes explorers that place very large regions early: a verbose early region can exhaust the budget before more useful evidence appears. The main experiments use $B = 500$ for the primary rank-aware score and additionally compute $B \in \{100, 300, 500\}$ in the released evaluator.

nDCG. For nDCG, each predicted region receives gain equal to the number of newly covered core lines. Regions are processed in predicted order, and the discounted gain is:

$$\text{DCG}@B = \sum_{i \in P_{\leq B}} \frac{g_i}{\log_2(i+2)}.$$

The normalized score is:

$$\text{nDCG}@B = \frac{\text{DCG}@B}{\text{IDCG}@B}.$$

Ideal-ordering for nDCG. The ideal DCG is computed under the same line budget as the explorer output. We construct the ideal order greedily: at each step, the evaluator selects the remaining ground-truth region with the largest marginal uncovered-line gain, subject to the remaining line budget. Ties are broken by shorter region length and then by repository path. This makes the normalization instance-specific and budget-matched: an explorer is compared against the best achievable ordering of the same target evidence, not against an unconstrained full-context oracle.

First useful hit. First Useful Hit (FUH) measures how early the explorer first surfaces any core evidence. Let i^* be the first predicted rank whose visible lines intersect Y . We define:

$$\text{FUH} = \begin{cases} 1 - i^*/|P|, & \text{when such } i^* \text{ exists,} \\ 0, & \text{otherwise.} \end{cases}$$

Higher values indicate that useful evidence appears earlier in the ranked list.

Efficiency and noise. Context efficiency is the fraction of predicted visible lines that fall inside either core or optional context:

$$\text{CtxEff} = \frac{|L(P) \cap (L(R_{\text{core}}) \cup L(R_{\text{opt}}))|}{|L(P)|}.$$

Noise rate is the fraction of predicted regions that overlap neither core nor optional context. The main table reports region-level noise.

Aggregation. Metrics are computed per instance and then averaged over instances. Empty predictions receive zero for coverage, ranking, first-hit, and efficiency metrics. Predictions with invalid paths or empty intervals are discarded before scoring.

D Restricted-Context Validation Protocol

This appendix supplements §3.4 and §4.2. The restricted-context protocol is used to test whether the selected regions can support actual patch generation under a fixed patching scaffold; it is not part of the standard upstream scoring loop.

Context materialization. For each explorer output, we first normalize paths, clip intervals to file boundaries, and remove invalid regions. For selected files, only the predicted line intervals remain visible. Lines outside the selected intervals are replaced with blank placeholders rather than deleted, so that repository paths and line numbers remain stable during patch generation and debugging. Files not selected by the explorer are hidden from the patching agent.

Fixed patch scaffold. All restricted-context runs use the same patcher, prompt template, tool set, and interaction budget. The only variable is the visible context produced by the explorer. This control is important because otherwise a higher resolve rate could reflect a stronger patch-generation scaffold rather than better exploration.

Patch application and harness evaluation. After patch generation, the predicted diff is applied to the original repository checkout and evaluated with the source benchmark’s executable harness. An instance is counted as resolved only when the patch passes the benchmark’s standard tests. Empty patches, unparsable diffs, failed patch applications, and test failures are all counted as unresolved.

Failure diagnostics. For every unresolved run, we log a coarse failure reason: no diff generated, invalid diff, patch failed to apply, patch outside visible context, applied patch failed tests, timeout, or infrastructure error. These diagnostics do not change the resolved/unresolved label; they are used to understand how restricted context affects repair.

E Explorer Implementation Details

This appendix supplements the explorer setup in §4.3. All methods are converted to the same output contract before scoring: an ordered list of at most $K = 5$ repository-relative line regions. Each region is represented as a path and a closed line interval. Invalid paths, empty intervals, and regions outside the repository checkout are discarded before evaluation.

Retrieval baselines. BM25 and TF-IDF use the issue statement as the query and rank repository chunks by lexical similarity. The top-ranked chunks are converted into line regions. Potion uses the same chunk-and-rank interface as a lightweight dense retrieval baseline.

Agentic explorers. For agentic explorers, each system is run under its original search or localization scaffold. We then normalize the resulting file, function, or region outputs into line-level regions. When a method produces file-level outputs, we map them to the most specific span supported by the method output or associated read trace. This conversion is applied before scoring so that all methods are compared under the same ranked-region contract.

Output validation. Before scoring, each prediction is checked for path validity, interval validity, and repository membership. Invalid predictions are dropped. Predictions with empty intervals are dropped. The remaining predictions are scored according to their original order.

F Case Study: `scikit-learn/scikit-learn#10844`

To make the quantitative results in §4.3 concrete, we walk through a single instance for which all 14 explorers produced output and whose ground truth is small enough to inspect end-to-end. We pick a real numerical-overflow bug whose ranking on this single instance closely tracks the ordering in the main tables, so the case mirrors — rather than distorts — the global picture.

Issue (scikit-learn/scikit-learn#10844, **from SWE-Bench Verified**). The issue is titled “*fowlkes_mallows_score* returns *RuntimeWarning* when variables get too big.” The reporter observes that the line

```
return tk / np.sqrt(pk * qk) if tk != 0. else 0.
```

inside `sklearn/metrics/cluster/supervised.py` silently overflows when `pk * qk` exceeds 2^{32} , because `pk` and `qk` are 32-bit integers; the result is a `RuntimeWarning` and a corrupted score. The fix casts the operands to `np.float64` before the multiplication, plus adds a regression test in `tests/test_supervised.py` that triggers the overflow path.

Ground truth. The refined ground truth lists two core files spanning only 26 lines in total, so the line-recall view is not dominated by wide whole-file scopes:

Path	Line range	Role
<code>sklearn/metrics/cluster/supervised.py</code>	850–870	modified function (<code>fowlkes_mallows_score</code>)
<code>sklearn/metrics/cluster/tests/test_supervised.py</code>	245–249	5-line regression test for the overflow case

Per-explorer outputs. Table 8 reports every explorer’s top-5 output and the resulting metrics. Regions are abridged to fit the column; entries in [brackets] mark a region whose file overlaps a ground-truth file.

Table 8: Top-5 outputs and metrics on `scikit-learn/scikit-learn#10844`. HF = HitFile, Noise = NoiseFile, Rec_ℓ = line recall, F1 = line F1, Cov = weighted core coverage.

Explorer	Returned regions (top-5, abridged)	HF↑	Noise↓	Rec_ℓ ↑	F1↑	Cov↑
Oracle	[supervised.py:850–870], [test_supervised.py:245–249]	1.00	0.00	1.00	1.00	1.00
Random	unrelated files from other repos	0.00	1.00	0.00	0.00	0.00
TF-IDF	ISSUE_TEMPLATE.md, CONTRIBUTING.md, doc/support.rst, doc/faq.rst	0.00	1.00	0.00	0.00	0.00
Potion (RAG)	ISSUE_TEMPLATE.md, CONTRIBUTING.md	0.00	1.00	0.00	0.00	0.00
BM25	(×2), doc/support.rst, sklearn/_init_.py ISSUE_TEMPLATE.md, PR_TEMPLATE.md, CONTRIBUTING.md (×2), [supervised.py:781–860]	0.50	0.75	0.42	0.07	0.31
AutoCodeRover	[supervised.py:787–859] (only 1 region emitted)	0.50	0.00	0.38	0.20	0.29
OrcaLoca	[supervised.py:787–859] (only 1 region emitted)	0.50	0.00	0.38	0.20	0.29
LocAgent	[supervised.py:787–859], [supervised.py:53–107], [supervised.py:34–50]	0.50	0.00	0.38	0.12	0.29
CoSIL	[supervised.py:1–∞], cluster/setup.py, metrics/setup.py, cluster/_init_.py, metrics/_init_.py	0.50	0.80	0.81	0.05	0.60
Claude Code	[supervised.py:28–31/53–107/787–859/193–214], [test_supervised.py:239–276]	1.00	0.00	0.58	0.14	0.69
Mini-SWE-Agent	[supervised.py:787–859/53–95], [test_supervised.py:239–276], doc/clustering.rst, doc/whats_new/v0.18.rst	1.00	0.50	0.58	0.14	0.69
AweAgent	[supervised.py:28–31/53–107/787–859], [test_supervised.py:239–276], doc/clustering.rst	1.00	0.33	0.58	0.15	0.69
Codex	[supervised.py:852–859/53–107/579–605], [test_supervised.py:239–276/170–184]	1.00	0.00	0.50	0.15	0.63
OpenHands	[supervised.py:787–859/53–107], [test_supervised.py:239–276/170–184], metrics/classification.py	1.00	0.33	0.58	0.14	0.69

Discussion. Three patterns are visible, and all three echo the global ordering reported in §4.3.

First, one-shot lexical retrieval is essentially useless on this bug. TF-IDF, Potion, and Random all reach HitFile = 0. The issue text contains no rare identifier that pins down the modified module: it

talks about `RuntimeWarning`, overflow, and integer multiplication, all of which are far more frequent in the project’s templates and documentation than in the implementation. BM25 partially recovers ($\text{HitFile} = 0.50$) by anchoring on `fowlkes_mallows_score` appearing both in the title and in the file, but it still buries the function below four template files and never reaches the regression test. This is the regime the global numbers describe: sparse retrievers cluster near Random on file hit, and Potion adds little above them.

Second, the academic localizers behave like the rest of the paper: precise on the implementation file, blind to the test file. AutoCodeRover, OrcaLoca, LocAgent, and CoSIL all reach `supervised.py` and land squarely on the `fowlkes_mallows_score` block, but none of them surfaces `test_supervised.py`; their HitFile therefore caps at 0.50. AutoCodeRover and OrcaLoca additionally under-fill the top-5 budget, emitting only one region, which costs them line recall and coverage even within the file they do reach. CoSIL again emits whole files, which lifts Rec_ℓ to 0.81 at the cost of high file-level noise — the same $\text{Rec}_\ell/\text{HitFile}$ asymmetry it shows in the main table.

Third, all five general-purpose agents converge on the same fix neighborhood. Claude Code, Mini-SWE-Agent, AweAgent, Codex, and OpenHands all reach both ground-truth files ($\text{HitFile} = 1.00$), and all land on the `fowlkes_mallows_score` body in `supervised.py:787-859` plus the surrounding test block in `test_supervised.py:239-276`. Differences between them reduce to small variations in span shape and how many auxiliary regions they include: Codex emits the tightest spans on the fix line itself (852-859), while AweAgent and OpenHands include extra context such as imports or related metrics. This mirrors the main-table observation that the general agents form a tight cluster on HitFile and Cov , with the remaining variance dominated by how much surrounding code each one chooses to return.

Takeaway. On this instance the practical ranking is $\text{Random/TF-IDF/Potion} \ll \text{BM25} < \text{academic localizers (one file only, precise)} < \text{general agents (both files, function-scoped spans)} < \text{CoSIL (recall by whole-file emission)} < \text{Oracle}$. The relative ordering and the magnitude of the gaps both match the global tables: the lexical baselines sit at the floor, the academic localizers occupy a narrow precision-leaning band, the general agents share a higher operating point on file hit and coverage, and only Oracle achieves matching scores on *both* line-level and file-level metrics. The case therefore illustrates why we report file hit, line recall, and coverage together: each metric independently reproduces a different part of this same ordering.

G Reproducibility, Compute, and Limitations

This appendix collects implementation information that is not central to the main argument but is needed to interpret the released artifact and compute cost.

Reproducibility. The released artifact contains benchmark records, the common explorer-output schema, metric computation scripts, and restricted-context validation scripts. Each instance records source provenance, repository metadata, and line-level context annotations. The evaluation pipeline consumes ranked-region predictions and produces both per-instance metrics and aggregate tables.

Compute. Sparse retrieval baselines run on CPU workers after repository indexing. Dense retrieval additionally requires embedding computation but no fine-tuning. Agentic explorers and restricted-context validation are the most expensive components because they require LLM calls and executable harness runs. For each downstream run, we log the model, prompt, tool budget, wall-clock time, patch status, and resolved status.

Limitations. SWE-Explore covers instances solved by at least one agent in our pool, so it does not represent the full distribution of unsolved repository-level issues. Its trajectory-derived ground truth is an empirical approximation of useful context, not a proof that no other evidence could support a valid solution. Some valid solution paths may rely on different evidence than the successful trajectories we observe. The restricted-context protocol should therefore be read as a controlled validation of exploration metrics, rather than as an absolute measure of patch-generation ability.

Responsible release. SWE-Explore is derived from public software-engineering benchmarks and public repository metadata. The release excludes private repositories, credentials, and user data.

Benchmark records preserve source attribution and include documentation for schema, provenance, and intended use.